

Contents

guide_Roofline: An Insightful Visual Performance Model for Floating-Point Programs and Multicore Architectures **1**

- 0. 这篇导读应该怎么用 1
- 1. 当时的背景: 多核时代缺一个简单模型 2
- 2. 核心定义: operational intensity 2
- 3. 一张 Roofline 图长什么样 3
- 4. 论文里的第一个例子: Opteron X2 和 X4 4
- 5. Roofline 不只是分类, 还能指导优化 4
- 6. Ceilings 怎样决定优化顺序 5
- 7. 把 3Cs model 接到 Roofline 5
- 8. 论文选择的 4 台机器 6
- 9. 论文选择的 4 个 kernel 7
- 10. 四个 kernel 的故事 7
 - 10.1 SpMV 7
 - 10.2 LBMHD 8
 - 10.3 Stencil 8
 - 10.4 3-D FFT 8
- 11. 实验证据链 8
- 12. 论文里的常见误解 9
- 13. 附录: Roofline 怎么量 9
- 14. 和 GPU/LLM 的直接连接 10
 - GEMM 10
 - LayerNorm 11
 - Attention 11
 - KV cache 11
- 15. 本仓库代码怎么对应论文 11
- 16. 一个张量级例子: GEMM 和 GEMV 为什么不同 12
- 17. 这篇论文的局限 13
- 18. 对今天学习 LLM infra 的意义 13
- 19. 新手复习问题 14
- 20. 一句话收束 14

guide_Roofline: An Insightful Visual Performance Model for Floating-Point Programs and Multicore Architectures

原论文: Roofline: An Insightful Visual Performance Model for Floating-Point Programs and Multicore Architectures

本地原文 PDF: [learning/gpu-architecture/paper/01_roofline_model.pdf](#)

作者: Samuel Williams, Andrew Waterman, David Patterson

论文语境: 2008 submitted/revised, later published as the classic Roofline model paper

0. 这篇导读应该怎么用

Roofline 是性能工程里最应该先学会的图。它不告诉你每个 cycle 发生了什么, 但它能很快回答一个关键问题:

这个 kernel 慢，是因为算力不够，还是因为数据喂不上？

对 LLM 全栈学习来说，这篇非常重要。你以后看 CUDA kernel、FlashAttention、fused layernorm、GEMM、MoE dispatch、KV cache 读写、batching、NVLink 通信，都要不断做类似判断：

- 这段计算的 FLOPs 有多少？
- 它从 HBM 或更低层 memory 搬了多少 byte？
- 它的 operations per byte 是否足够高？
- 如果不够高，优化 peak TFLOPS 没意义，先减少 memory traffic。
- 如果足够高，才开始认真看 tensor core、SIMD、warp-level scheduling、occupancy。

注意，论文原文使用的是 operational intensity，不是泛泛的 arithmetic intensity。它定义的是 operations per byte of DRAM traffic，也就是 cache hierarchy 已经过滤后，真正打到主存的流量。这个细节非常关键。两个 kernel 可能做同样多 FLOPs，但一个 cache locality 好、DRAM traffic 少，它的 operational intensity 就更高。

1. 当时的背景：多核时代缺一个简单模型

论文开头的时代背景是 2008 年左右的 multicore 转向。过去很多 CPU 架构虽然品牌和 ISA 不同，但设计哲学相似：cache、pipeline、superscalar、out-of-order execution。程序员和编译器作者虽然辛苦，但至少面对的是一个相对稳定的主流设计范式。

进入 multicore 之后，这种稳定性被打破。不同厂商开始探索完全不同的设计：

- 少量复杂核心。
- 很多简单核心。
- 大量硬件线程。
- 本地存储替代 cache。
- SIMD 宽度和 memory controller 组织各不相同。

这会让程序员非常痛苦。一个 kernel 在某台机器上快，在另一台机器上慢，原因可能是 cache、memory bandwidth、instruction mix、SIMD、prefetch、NUMA affinity、load balance 中的任何一个。

作者认为需要一个类似 cache 3Cs model 的东西。3Cs model 并不完美，它把 cache miss 分成 compulsory、capacity、conflict，忽略了很多细节，但它有洞察力，能指导人优化。Roofline 的定位也是如此：

不是精确预测器

不是 cycle-level simulator

而是一个 bound-and-bottleneck 模型

它的目标是用一张图告诉程序员、编译器作者和架构师：

- 当前 kernel 的上界在哪里。
- 主要瓶颈是 memory 还是 compute。
- 优化应该先从哪类动作开始。
- 一台机器的 peak FLOPS 对实际 workload 是否容易达到。

2. 核心定义：operational intensity

论文刻意不用传统的 arithmetic intensity 或 machine balance，而是定义：

$$\text{operational intensity} = \text{useful operations} / \text{DRAM bytes accessed}$$

这里的 DRAM bytes 指的是 cache hierarchy 过滤之后，cache 和主存之间的 traffic，不是 processor 和 L1/L2 cache 之间的全部 traffic。

这句话一定要读慢一点。假设一个 kernel 做 $1e12$ 次 operation:

```
case A:
  DRAM traffic =  $1e12$  bytes
  OI = 1 operation / byte
```

```
case B:
  DRAM traffic =  $1e10$  bytes
  OI = 100 operations / byte
```

两者 FLOPs 一样, 但 case B 更可能接近 compute roof, 因为它把数据复用得更好。这就是为什么 blocking、tiling、shared memory、cache locality、no-allocate store、compression 都会影响 Roofline 位置。它们不一定改变 operation 数, 但会改变打到 DRAM 的 byte 数。

对 GPU 学习来说, 可以先把 OI 翻译成一句话:

每从 HBM 搬 1 byte, 我能做多少有效计算?

GEMM 的 OI 很高, 因为 A/B tile 读进来后会被重复使用。LayerNorm、embedding lookup、小 batch GEMV、某些 reduce 的 OI 很低, 因为数据读一遍做很少计算就结束了。

3. 一张 Roofline 图长什么样

Roofline 图是 log-log 图:

y-axis: attainable performance, such as GFLOP/s or TFLOP/s

x-axis: operational intensity, operations per DRAM byte

它有两条基本上界:

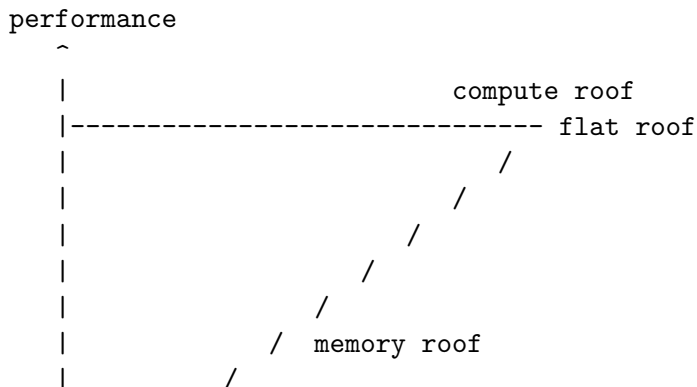
```
compute roof:
  horizontal line = peak compute
```

```
memory roof:
  slanted line = peak memory bandwidth * operational intensity
```

核心公式是:

```
attainable performance
= min(peak compute performance,
      peak memory bandwidth * operational intensity)
```

用 ASCII 画出来大概是:



+-----> operational intensity
ridge point

当一个 kernel 的 OI 落在 ridge point 左边，它撞到斜屋顶，是 memory-bound。当 OI 落在 ridge point 右边，它撞到水平屋顶，是 compute-bound。

ridge point 的公式是：

$$\text{ridge point} = \text{peak compute} / \text{peak memory bandwidth}$$

这表示要达到 peak compute，kernel 至少需要多少 operations per byte。ridge point 越靠右，说明机器越“偏算力”，程序越难喂饱算力。ridge point 越靠左，说明 memory bandwidth 相对充足，更多 kernel 有机会接近 peak。

4. 论文里的第一个例子: Opteron X2 和 X4

论文用 AMD Opteron X2 做第一张图。这个系统峰值 double precision performance 约 17.6 GFlops/s，作者用 microbenchmark 测到可持续 memory bandwidth 约 15 GB/s。因此它的 ridge point 大约在 1 operation/byte 附近。

然后作者比较了 Opteron X4。X4 核心数更多，单核浮点能力也更强，峰值 compute 大幅提高；但因为它和 X2 共享类似 socket/memory channel 约束，memory bandwidth 没有按同样比例提高。结果 ridge point 从约 1.0 移到约 4.4。

这说明什么？

compute peak 增长很快
memory bandwidth 没有同速增长
ridge point 右移
更多 kernel 会卡在 memory roof

这正是后来 GPU/LLM 时代反复出现的问题。现代 GPU 的 tensor core peak TFLOPS 很吓人，但 HBM bandwidth 没有同等倍数增长。很多 LLM op 看起来在 GPU 上跑，却根本没有接近 peak TFLOPS，因为它们的 OI 太低。

5. Roofline 不只是分类，还能指导优化

只知道 memory-bound 或 compute-bound 还不够。论文的第二个关键贡献是 ceilings。

Roofline 的基本两条线告诉你理论上限，但如果程序离上限很远，你还要知道：

我被哪个优化缺失挡住了？

先做哪个优化收益最大？

作者把不同优化看成 roofline 下面的一层层 ceiling。你没有做某个优化，就突破不了对应 ceiling。gap 越大，说明做这个优化的潜在收益越大。

论文把优化分成 compute ceilings 和 memory bandwidth ceilings。

compute 侧的典型 ceilings：

- Improve ILP and apply SIMD。
- Balance floating-point operation mix。
- 对有 FMA 或加乘并行 datapath 的机器，保持 add/multiply 平衡。

memory 侧的典型 ceilings：

- Restructure loops for unit stride access。

- Ensure memory affinity.
- Use software prefetching.

这套思想迁移到 GPU/LLM 上，可以对应成：

- compute ceiling: tensor core 没用上、MMA shape 不合适、warp divergence、instruction mix 差、occupancy 太低。
- memory ceiling: global memory coalescing 差、HBM traffic 太大、shared memory tiling 不够、L2 reuse 低、KV cache 访问不连续、cross-GPU traffic 太重。

6. Ceilings 怎样决定优化顺序

论文强调，ceilings 的顺序不是纯理论顺序，而是“从下到上”排：

低 ceiling：

编译器容易做，或程序员低成本能做

高 ceiling：

更困难，或 kernel 本身未必具备

如果一个 kernel 的 vertical line 落在某个区域，Roofline 会告诉你应该优先做哪类优化。举个简化图：

memory-bound kernel：

```
current point
|
v
memory ceiling: no unit stride
memory ceiling: no affinity
memory ceiling: no prefetching
main memory roof
compute roof
```

这时你先不要纠结 peak FLOPS。先问：

- 有没有减少 DRAM traffic 的机会？
- 访问是否 unit stride/coalesced？
- 数据是不是放在离计算单元近的位置？
- prefetch 或 async copy 能不能隐藏 latency？

如果是 compute-bound kernel，则先问：

- 是否使用 SIMD/tensor core？
- 是否有足够 ILP？
- operation mix 是否平衡？
- occupancy 是否被 register 或 shared memory 限制？

这就是 Roofline 比一句“memory-bound”更有用的地方。它不只是贴标签，而是给优化动作排序。

7. 把 3Cs model 接到 Roofline

论文第五节把 cache 3Cs 和 operational intensity 接起来。

3Cs 是：

- compulsory misses: 第一次必须访问的数据。

- capacity misses: cache 装不下工作集导致的 miss。
- conflict misses: 映射冲突导致的 miss。

Operational intensity 的分母是 DRAM traffic。如果你消除 capacity/conflict misses, DRAM bytes 就减少, OI 会右移。

用图表示:

before cache optimization:

operations = fixed
DRAM bytes = high
OI = low
point is left

after blocking/padding/no-allocate store:

operations = roughly fixed
DRAM bytes = lower
OI = higher
point moves right

论文给的例子包括:

- padding arrays, 减少 cache line conflict。
- no-allocate store, 避免写入会覆盖的数据时还先把 cache line 读进来。
- 对 3-D FFT, 工作集/plane 能否放进 cache 会影响 OI。

对 GPU 来说, 对应的现代版本是:

- shared memory tiling。
- register tiling。
- L2-friendly layout。
- memory coalescing。
- fused kernels 减少中间写回 HBM。
- FlashAttention 这类 IO-aware algorithm, 直接减少 HBM traffic。

所以你读 FlashAttention 时, 其实已经在用 Roofline 的语言:

不改变 exact attention 结果

减少 HBM reads/writes

提高 operational intensity

把 kernel 从 memory-bound 推向更高 roof

8. 论文选择的 4 台机器

为了展示模型, 作者选了 4 种很不一样的 multicore:

- Intel Xeon Clovertown e5345。
- AMD Opteron X4 Barcelona 2356。
- Sun UltraSPARC T2+ Niagara 2。
- IBM Cell QS20。

它们的设计差异非常大:

- Xeon 有较高 peak compute, 但 front side bus 和 coherency traffic 让 memory 行为复杂。
- Opteron X4 有 on-chip memory controller 和 HyperTransport, memory behavior 比 Xeon 更容易理解。

- T2+ 有很多简单核心和大量硬件线程, memory bandwidth 很强, ridge point 很低。
- Cell 有 PowerPC core 加 SPE, 本地存储替代 cache, 需要 DMA 显式搬数据。

论文里的结论很有启发:

最高 peak FLOPS 不等于最好优化

最低 ridge point 的机器往往更容易接近自身 peak

T2+ 的 peak compute 不最高, 但因为 ridge point 低, 许多 kernel 更容易达到较高比例的峰值。Xeon peak compute 高, 但 ridge point 高, 很多低 OI kernel 很难喂饱它。

迁移到 GPU:

一张 GPU 的 tensor core TFLOPS 很高, 不代表你的 kernel 能接近它。

如果 ridge point 很高, 低 OI op 会长期卡在 HBM 或 cache traffic 上。

9. 论文选择的 4 个 kernel

作者没有随便挑 benchmark, 而是从 Berkeley Seven Dwarfs 的思想出发, 挑了 4 个重要 数值 kernel:

1. SpMV, sparse matrix-vector multiply。
2. LBMHD, Lattice-Boltzmann Magnetohydrodynamics。
3. Stencil, 3-D stencil。
4. 3-D FFT。

论文给出的 operational intensity 大致范围:

- SpMV: 0.17 到 0.25。
- LBMHD: 0.70 到 1.07。
- Stencil: 0.33 到 0.50。
- 3-D FFT: 1.09 到 1.64。

这些数都不高。也就是说, 在作者研究的那些机器上, 大部分 case 都很容易 memory-bound。论文总结 Table 4 时说, 16 个 kernel-computer 组合里, OI 从 0.25 到 1.64, 中位数 约 0.60。对 ridge point 高达 4.4 或 6.7 的 x86 系统来说, 这些 kernel 很难接近 peak compute。

这正是 Roofline 的力量: 它能提前告诉你, 不要被 peak GFLOPS 迷惑。kernel 的位置 在图上很左时, 先看 memory。

10. 四个 kernel 的故事

10.1 SpMV

SpMV 是:

$$y = A * x$$

其中 A 是 sparse matrix, x 和 y 是 dense vector。SpMV 只有非零元素参与计算, 但内存访问不规则, 索引和数据结构开销大, 所以常常低于 peak 的 10 percent。

论文里 SpMV 的 OI 从 0.17 提高到 0.25, 提升来自 register blocking 等优化。即使如此, 它仍然在所有机器 ridge point 左侧, 所以优化主要围绕 memory system。

LLM 类比:

- embedding lookup。
- 稀疏专家路由。

- batch 很小的 GEMV。
- 大量 gather/scatter。

这些 op 常常不是算不动，而是数据访问太散。

10.2 LBMHD

LBMHD 是结构化网格类模拟，数据结构复杂、memory access irregular。论文提到 no-allocate store 可以把 OI 从 0.70 提高到 1.07，因为它减少了 write-allocate 导致的额外 memory traffic。

LLM 类比:

- 某些 fused op 的中间张量如果总是写回 HBM，再读回来，OI 会很低。
- 如果通过 fusion 或写入策略减少中间 traffic，点会向右移动。

10.3 Stencil

Stencil 用附近点更新当前点。论文里的 3-D stencil 对一个 256 cube 网格，每个点用 附近 6 个方向的邻居和自身。它每 24 bytes compulsory memory traffic 做 8 次浮点操作，所以 OI 大约 0.33，在 write-allocate 架构下很容易 memory-bound。

GPU 类比:

- 卷积和局部 stencil 如果没有 shared memory tiling，就会重复读邻域。
- 加 tiling 后复用提高，DRAM traffic 降低，OI 右移。

10.4 3-D FFT

FFT 的 OI 会随问题大小和 cache 行为变化。论文报告 128 cube 和 512 cube 的 3-D FFT OI 约 1.09 到 1.41；在某些机器上，128 cube 的一个 plane 能放进 cache，OI 可到 1.64。

FFT 的学习价值是: operational intensity 不一定是固定常数。它依赖 problem size、cache hierarchy 和 implementation。

LLM 类比:

- attention 的 OI 随 sequence length、head dim、tiling 策略变化。
- batch size 改变，GEMM 从 GEMV-like 变成 GEMM-like，OI 会大幅变化。

11. 实验证据链

论文不是只画一张示意图，而是对 4 台机器 x 4 个 kernel 做 Roofline 分析，并用 ceilings 解释优化后性能。

证据链可以按这个顺序读。

第一步，作者为每台机器测量或计算基础 roofs:

- peak floating-point performance。
- sustainable DRAM bandwidth。
- ridge point。

第二步，为每个 kernel 估计 operational intensity。注意它是 kernel 和 architecture 共同决定的，因为 cache behavior 会影响 DRAM traffic。

第三步，在图上画 kernel 的 vertical line。红色点表示 achieved performance，它应该落在 roofline 和相应 ceilings 之下。

第四步，用 ceilings 解释优化空间。比如 memory affinity、unit-stride、prefetch、SIMD、operation mix，分别对应不同上界。

第五步，总结 Table 4。论文说所有 16 个 case 都符合这个 bound-and-bottleneck 模型：实际 achieved performance 被相邻的 upper/lower ceilings 夹住，优化方向也和较低 ceiling 的建议一致。

这不是“模型精确预测每个数值”，而是证明：

Roofline 能把实际性能限制在合理上界内

ceilings 能解释为什么某些优化有效

ridge point 能解释为什么某些机器更容易写快

12. 论文里的常见误解

论文第七节专门回答了一组 fallacies，这部分非常适合新手。

误解一：Roofline 没考虑 cache 和 prefetch。

论文回答：OI 的 DRAM bytes 已经是 cache hierarchy 过滤后的 traffic。memory bandwidth roof 也可以包含 prefetch、blocking 等优化后的可持续带宽。ceilings 还可以显式表示没有 prefetch 或没有 affinity 的上界。

误解二：cache 加大一定提高 OI。

论文回答：不一定。如果 kernel 已经接近 compulsory traffic，增大 cache 没用。cache 只对 capacity/conflict misses 有帮助。对某些 3-D FFT problem size，cache 能放下 plane，才会明显提高 OI。

误解三：Roofline 没考虑 memory latency。

论文回答：没有 prefetch 的 bandwidth ceilings 本质上就反映了不能隐藏 latency 的损失。要推高 memory roof，需要足够并发和预取。

误解四：Roofline 只适合 floating-point。

论文回答：不必。你可以把 y-axis 换成其它 rate，比如 exchanges per second、frames per second、sorts per second；x-axis 也可以换成某种 operation per byte。论文用 3-D FFT transpose phase 举例，它没有 floating-point operation，也可以用 exchange 作为 performance metric。

误解五：每个 kernel 都要重新计算整张 Roofline。

论文回答：machine roofs 和 ceilings 通常每台机器每个 metric 测一次即可。每个 kernel 需要重新估计的是 operational intensity。

13. 附录：Roofline 怎么量

论文附录 A 解释了如何构造 Roofline。核心需要三类数字：

1. operational intensity * bandwidth
2. in-core flop/sec
3. in-core flop/sec as a function of floating-point fraction

Operational intensity 可以用 performance counters 测 operation 和 DRAM traffic；有些 kernel 也可以手算 operation 和 minimum traffic，从而得到上下界。

Memory bandwidth 不能只迷信 STREAM 的单一数字。作者写了更细的 microbenchmark, 包括 dot product 和 copy, 并逐步加入:

- padding, 避免 bank/cache conflict。
- cache bypass 或修正 write-allocate traffic。
- memory affinity。
- software prefetch with tuned distance。
- snoop filter effectiveness。

In-core parallelism 也有 ceilings。作者从一个 reduction 例子出发:

$y = x[1] + x[2] + \dots + x[N]$

如果每个 thread 只是做 dependent scalar add chain, 就暴露 floating-point pipeline latency。加入 unrolling 和多个 partial sums 可以提高 ILP。再加入 SIMD, 可以提高 data-level parallelism。如果改成 multiply-add 形式, 还要看 add/multiply 或 FMA 是否平衡。

这些细节迁移到 GPU 上, 可以读成:

- reduction 是否有足够 parallelism。
- warp 内是否有 dependency chain。
- 是否用到 vectorized load/store 或 tensor core。
- instruction mix 是否被 address calculation、predicate、integer op 拖住。
- memory queues 是否有足够 in-flight requests 满足 Little's Law。

14. 和 GPU/LLM 的直接连接

论文原本是 multicore CPU 语境, 但它在今天的 GPU 上更常用。只要把单位换成:

peak compute:

TFLOP/s for FP32, BF16, FP8, FP4, tensor core, sparse tensor core

memory bandwidth:

HBM TB/s, L2 bandwidth, shared memory bandwidth, NVLink bandwidth

operational intensity:

useful FLOPs per HBM byte, or per L2 byte, or per network byte

就可以分析很多 LLM op。

GEMM

大矩阵乘法:

$C = A @ B$

FLOPs roughly = $2 * M * N * K$

bytes roughly = $\text{dtype_bytes} * (M*K + K*N + M*N)$

M/N/K 都大时, A 和 B tile 会被重复使用, OI 很高, 容易 compute-bound。此时优化重点是 tensor core、MMA shape、tile size、warp scheduling、occupancy。

小 batch GEMV:

$M = 1$

N and K large

这时复用低, OI 低, 通常 memory-bound。你换更高 peak TFLOPS 的 GPU, 不一定带来 线性提升。

LayerNorm

LayerNorm 对每个元素做有限操作, 但要读写大张量。它通常 OI 低, memory-bound。优化方向是 fusion、vectorized load/store、减少 HBM pass、让访存连续。

Attention

naive attention 会产生或读取大 $S \times S$ score matrix, HBM traffic 很重。FlashAttention 的关键是 IO-aware tiling:

减少 HBM reads/writes

保持 exact attention

提高 effective operational intensity

这就是 Roofline 思维在 LLM 里的漂亮应用。

KV cache

decode 阶段每步读历史 KV。FLOPs 不一定高, 但 KV cache bytes 随上下文增长。长上下文 decode 很容易 memory-bound 或 bandwidth-bound。优化要看 cache layout、paged KV、batching、quantized KV、prefetch 和 memory coalescing。

15. 本仓库代码怎么对应论文

本专题代码分成几层。

common.py 提供现代 GPU 的 peak compute 和 HBM bandwidth, 并计算 ridge point:

```
def ridge_point_bf16(self):  
    return bf16_tflops / hbm_tb_s
```

roofline.py 把 LLM 常见 op 做成 profile:

```
gemm_profile(m, n, k)  
attention_profile(b, h, s, d)  
layernorm_profile(n_tokens, hidden)
```

它会计算:

```
ai = flops / bytes_moved  
achievable = min(peak_tflops, hbm_tb_s * ai)  
bound_by = compute or memory
```

roofline_original_minimal.py 是这次新增、最贴原论文模型。它使用论文术语:

- MachineRoofline
- KernelObservation
- Ceiling
- operational_intensity
- ridge_point
- attainable_with_ceilings
- recommend_optimizations

你可以运行:

```
.\.venv\Scripts\python.exe learning\gpu-architecture\src\roofline_original_minimal.py
```

也可以跑本专题测试:

```
.\.venv\Scripts\python.exe learning\gpu-architecture\src\tests\test_all.py
```

memory_hierarchy.py 让你把 register、shared memory、L2、HBM 放到同一个层级里。Roofline 里用哪个 bandwidth, 要看你关心的是 DRAM/HBM roof、L2 roof 还是 shared memory roof。

tensor_core.py 连接 compute roof。不是所有 matmul 都自然打到 peak; 要用对 MMA shape、dtype 和 tile。

sm_occupancy.py 连接 ceilings。一个 kernel 可能理论上 compute-bound, 但因为 register、shared memory 或 block scheduling 限制, 实际被 occupancy ceiling 卡住。

capstone_roofline_zoo.py 把多个 LLM-like op 放到 A100/H100/H200/B200 上比较。它的价值不是给出生产性能, 而是训练直觉:

哪些 op 随 GPU peak compute 增长明显?

哪些 op 主要被 HBM bandwidth 限制?

H100 到 B200 的变化对不同 op 是否一样?

16. 一个张量级例子: GEMM 和 GEMV 为什么不同

假设 BF16, dtype bytes = 2。

大 GEMM:

$M = 4096, N = 4096, K = 4096$

$\text{FLOPs} = 2 * M * N * K$

$\text{bytes} = 2 * (M * K + K * N + M * N)$

OI is very high

直觉是 A 的一个 tile 会和 B 的很多 tile 复用, B 的 tile 也会被多个 output tile 复用。数据搬一次, 做很多 multiply-add。

GEMV-like:

$M = 1, N = 4096, K = 4096$

$\text{FLOPs} = 2 * M * N * K$

$\text{bytes} = 2 * (M * K + K * N + M * N)$

OI is much lower

这里 B 的大矩阵读进来后复用少, M 太小导致输出维度不能充分摊薄 memory traffic。decode 阶段小 batch 线性层常常接近这个世界。

这解释了一个 LLM serving 里的常见现象:

prefill:

batch/sequence matrix 更大

GEMM-like

更容易用上 tensor core

decode:

每步 token 少

GEMV-like
更容易 memory-bound

所以看 serving 系统时, Roofline 要和 workload phase 一起读。

17. 这篇论文的局限

第一, Roofline 是上界模型, 不是精确 runtime predictor。它不能替代 profiler, 也不能告诉你每个 stall cycle 的原因。

第二, 论文的基本图是 steady-state 思维。真实 kernel 还有 launch overhead、tail effect、cache warm-up、scheduler behavior、branch divergence、bank conflict、pipeline bubble。

第三, OI 的估计不总是简单。尤其是现代 GPU, 有 L2 cache、shared memory、sector load、writeback、compression、tensor core pipe、async copy、TMA。你需要 profiler 或认真手算。

第四, Roofline 的结论依赖 metric。你用 HBM roof, 会得到 HBM-bound 解释; 你用 L2 roof, 可能得到另一个 bottleneck; 你用 NVLink byte, 甚至可以分析 communication roofline。

第五, LLM workload 有动态性。batch size、sequence length、KV cache length、dtype、parallelism strategy 都会移动点的位置。

18. 对今天学习 LLM infra 的意义

Roofline 是把 “感觉这个 op 很慢” 变成 “知道从哪里优化” 的最短路径。

它会训练你少说这种话:

这个 GPU 有 1000 TFLOPS, 为什么我的 kernel 没有 1000 TFLOPS?

多说这种话:

这个 kernel 的 OI 是多少?

它在 HBM roof 左边还是右边?

如果 memory-bound, 有没有减少 DRAM traffic 或提高 coalescing 的办法?

如果 compute-bound, tensor core、ILP、occupancy、instruction mix 哪个 ceiling 最低?

对 AI agent 学习也一样。不要让 agent 只回答 “优化 CUDA 有哪些技巧”。你应该让它 带你做诊断:

请根据 FLOPs 和 bytes 估算这个 op 的 operational intensity。

给定 H100 BF16 peak 和 HBM bandwidth, 判断它的 roofline bound。

如果它 memory-bound, 列出三种能减少 HBM traffic 的改法。

如果它 compute-bound, 列出三个可能的 compute ceilings。

这样学, 知识会进入脑子, 因为你每次都在做同一个闭环:

张量形状

- > FLOPs
- > bytes
- > operational intensity
- > roofline bound
- > bottleneck
- > optimization order
- > code experiment

19. 新手复习问题

读完后，请你不用看原文，回答下面问题：

1. 为什么论文用 operational intensity，而不是普通 arithmetic intensity？
2. Roofline 的 y-axis 和 x-axis 分别是什么？
3. 为什么 memory roof 是一条斜线？
4. ridge point 的公式是什么，它为什么能衡量“达到 peak 的难度”？
5. Opteron X4 为什么 ridge point 比 X2 右移？
6. ceilings 和基本 roofline 有什么区别？
7. 为什么要先提高 OI，再追求 compute peak？
8. cache 变大什么时候能提高 OI，什么时候不能？
9. SpMV 为什么通常低于 peak 很多？
10. 对 LLM decode 阶段，为什么 GEMV-like op 常常 memory-bound？

如果这些问题能讲清楚，你再去看看 CUDA kernel profiler，看到 memory throughput、SM utilization、tensor core utilization、L2 hit rate、dram bytes，就不会只是一堆数字。

20. 一句话收束

Roofline 的核心不是画一张漂亮图，而是强迫你把性能问题拆成两个单位：operations 和 bytes。只要你能估算这两个量，就能知道一个 kernel 该先减少数据流量，还是先追求更高的计算利用率。